

R2.09 — Initiation au développement Web

Durée : 2h · Travail en binôme · Pré-requis : TD3 terminé (TaskList, TaskForm, TaskCard fonctionnels)

 Objectifs du TD	
Menu de navigation	Barre de navigation responsive avec liens actifs et déconnexion
Page profil	Afficher et modifier les informations du compte via Supabase Auth
Changer de mot de passe	Mettre à jour le mot de passe de manière sécurisée
Upload d'avatar	Stocker une photo de profil dans Supabase Storage
Envoi de mails avec Resend	Envoyer un e-mail depuis React via une API Route Vercel

Partie A — Vérification du TD3 (10 min)

Rappel TD3

Votre app doit afficher dans le dashboard les tâches chargées depuis Supabase.

Le formulaire de création insère bien en BDD (table tasks) et les badges de priorité / statut / catégorie s'affichent sur TaskCard.

Si l'une de ces fonctionnalités est incomplète, réglez-la avant de continuer.

A.1 Nouveaux fichiers à créer dans ce TD

Votre arborescence src/ à l'issue de ce TD :

```
src/  
  lib/supabase.js  
  pages/  
    | LoginPage.jsx  
    | DashboardPage.jsx      ← sera enrichi (menu intégré)  
    | ProfilePage.jsx        ← à créer (Partie B)  
  components/  
    | Navbar.jsx             ← à créer (Partie A)  
    | TaskCard.jsx  
    | TaskForm.jsx  
    | TaskList.jsx  
  api/                       ← dossier Vercel serverless  
    | send-email.js          ← à créer (Partie C)  
  App.jsx                    ← sera mis à jour (route /profile)
```

Partie B — Menu de navigation (35 min)

Actuellement, votre app n'a pas de barre de navigation. Dans cette partie, vous allez créer un composant `Navbar.jsx` réutilisable, l'intégrer dans `DashboardPage` et `App.jsx`, et ajouter un indicateur de lien actif.

B.1 Ajouter la route `/profile` dans `App.jsx`

Avant de créer le menu, ajoutez la nouvelle route dans votre routeur. Modifiez `src/App.jsx` :

```
// src/App.jsx - ajoutez ces deux lignes en haut
import ProfilePage from './pages/ProfilePage';

// Dans <Routes>, ajoutez après la route /dashboard :
<Route path='/profile'
  element={session ? <ProfilePage session={session} /> : <Navigate to='/login' />} />
```



Route protégée

Même logique que `/dashboard` : si l'utilisateur n'est pas connecté et tente d'accéder à `/profile`, il est automatiquement redirigé vers `/login`.

B.2 Créer le composant `Navbar.jsx`

Créez `src/components/Navbar.jsx`. Ce composant utilise `useLocation()` pour détecter la page active et mettre le lien en surbrillance.

```
// src/components/Navbar.jsx

import { Link, useLocation } from 'react-router-dom';
import { supabase } from '../lib/supabase';

export default function Navbar({ session }) {

  const location = useLocation(); // URL courante ex: '/dashboard'

  async function handleLogout() {

    await supabase.auth.signOut();

    // App.jsx détecte session=null → redirige /login automatiquement
  }

  // Fonction utilitaire : renvoie le style du lien selon s'il est actif
  function linkStyle(path) {

    const isActive = location.pathname === path;

    return {

      padding: '0.5rem 1rem',

      borderRadius: '6px',

      textDecoration: 'none',

      fontWeight: isActive ? 700 : 400,

      background: isActive ? 'rgba(255,255,255,0.2)' : 'transparent',

      color: 'white',

    };
  }

  return (

    <nav style={{

      background: '#1A8C82',

      padding: '0 2rem',

      display: 'flex',

      alignItems: 'center',

      justifyContent: 'space-between',

      height: '56px',

      boxShadow: '0 2px 8px rgba(0,0,0,0.15)',

    }}>

```

B.3 Intégrer Navbar dans les pages

Modifiez `src/pages/DashboardPage.jsx` pour utiliser Navbar au lieu de l'ancien header :

```
// src/pages/DashboardPage.jsx - remplacez l'import et le <header>
import Navbar from '../components/Navbar';

export default function DashboardPage({ session }) {
  // ...états et fonctions inchangés
  return (
    <div style={{ minHeight: '100vh', background: '#F8FAFC' }}>
      <Navbar session={session} />    {/* ← remplace l'ancien <header> */}
      <main style={{ padding: '2rem' }}>
        {/* ...onglets et contenu inchangés */}
      </main>
    </div>
  );
}
```

Test

Lancez `npm run dev`. Naviguez entre `/dashboard` et `/profile` (la page 404 est normale pour l'instant).

Le lien actif doit apparaître en surbrillance blanche. La déconnexion depuis la Navbar doit renvoyer sur `/login`.

Partie C — Gestion du compte utilisateur (40 min)

Supabase Auth expose des méthodes pour lire et mettre à jour le profil de l'utilisateur connecté. Dans cette partie, vous créez une page de profil complète avec trois fonctionnalités : modifier son nom, changer son mot de passe et uploader un avatar.

C.1 Méthodes Supabase Auth utiles

Méthode SDK Supabase	Ce qu'elle fait
<code>supabase.auth.getUser()</code>	Récupère l'utilisateur courant et ses métadonnées.
<code>supabase.auth.updateUser({ data })</code>	Met à jour les métadonnées du profil (nom, avatar_url...).
<code>supabase.auth.updateUser({ password })</code>	Change le mot de passe de l'utilisateur connecté.
<code>supabase.storage.from('avatars').upload()</code>	Upload un fichier dans le bucket Supabase Storage.
<code>supabase.storage.from('avatars').getPublicUrl()</code>	Génère l'URL publique d'un fichier uploadé.

C.2 Configurer le bucket Supabase Storage

Avant de coder l'upload, créez un bucket de stockage dans Supabase :

1. Dans Supabase, allez dans Storage → New bucket.
2. Nom : avatars | Type : Public (pour que les URLs soient accessibles sans auth).
3. Cliquez Create bucket.
4. Dans Politiques → New policy → For full customization, ajoutez une politique INSERT/SELECT pour les utilisateurs authentifiés.

```
-- Politique SQL à coller dans l'éditeur de politique Supabase
-- Autoriser tout utilisateur authentifié à uploader et lire ses avatars

CREATE POLICY "upload own avatar"
ON storage.objects FOR INSERT
WITH CHECK (bucket_id = 'avatars' AND auth.role() = 'authenticated');

CREATE POLICY "read all avatars"
ON storage.objects FOR SELECT
USING (bucket_id = 'avatars');
```

C.3 Créer ProfilePage.jsx

Créez `src/pages/ProfilePage.jsx`. La page comporte trois sections : informations générales, changement de mot de passe, et avatar.

```
// src/pages/ProfilePage.jsx

import { useState, useEffect } from 'react';
import { supabase } from '../lib/supabase';
import Navbar from '../components/Navbar';

export default function ProfilePage({ session }) {
  const user = session.user;

  // — États : infos générales —————
  const [fullName, setFullName] = useState(user.user_metadata?.full_name || '');
  const [infoMsg, setInfoMsg] = useState('');
  const [infoErr, setInfoErr] = useState('');

  // — États : mot de passe —————
  const [newPass, setNewPass] = useState('');
  const [passMsg, setPassMsg] = useState('');
  const [passErr, setPassErr] = useState('');

  // — États : avatar —————
  const [avatarUrl, setAvatarUrl] = useState(user.user_metadata?.avatar_url || '');
  const [uploading, setUploading] = useState(false);

  // — Sauvegarder le nom —————
  async function handleSaveInfo(e) {
    e.preventDefault();
    setInfoErr(''); setInfoMsg('');
    const { error } = await supabase.auth.updateUser({
      data: { full_name: fullName } // stocké dans user_metadata
    });
    if (error) setInfoErr(error.message);
    else setInfoMsg('✅ Profil mis à jour !');
  }
}
```




user_metadata

Supabase stocke des données libres dans user_metadata (objet JSON). Vous pouvez y mettre ce que vous voulez : full_name, avatar_url, theme, langue...

Pour les lire : `session.user.user_metadata.full_name`

Pour les écrire : `supabase.auth.updateUser({ data: { full_name: 'Alice' } })`

C.4 Test de la page profil

5. Créez un compte ou connectez-vous.
6. Allez sur /profile via le lien 'Mon profil' dans la Navbar.
7. Modifiez votre nom complet → cliquez Sauvegarder → message de confirmation affiché.
8. Uploadez une photo → elle doit s'afficher dans le cercle avatar.
9. Changez votre mot de passe → vérifiez en vous déconnectant et reconnectant.

Partie D — Envoi de mails avec Resend (35 min)

Resend est un service d'envoi d'e-mails moderne, conçu pour les développeurs. Il fournit une API simple et un SDK JavaScript. Comme les clés API ne doivent jamais être exposées côté client, on utilise une API Route Vercel (fonction serverless) comme intermédiaire.



Pourquoi une API Route et pas directement dans React ?

Le SDK Resend contient votre clé API secrète. Si vous l'utilisiez directement dans un composant React, la clé serait visible dans le code source de la page par n'importe quel visiteur.

Solution : une API Route Vercel est une fonction qui s'exécute côté serveur — la clé reste secrète, seul le résultat (succès/erreur) revient au navigateur.

D.1 Créer un compte Resend et une clé API

10. Allez sur <https://resend.com> et créez un compte gratuit (100 mails/jour offerts).
11. Dans le dashboard Resend → API Keys → Create API Key.
12. Nommez-la 'kanban-rt' et conservez la valeur — elle ne s'affiche qu'une seule fois.
13. Ajoutez-la dans votre fichier .env.local :

```
# .env.local - ajoutez cette ligne  
RESEND_API_KEY=re_XXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

14. Ajoutez aussi la variable sur Vercel : Settings → Environment Variables → RESEND_API_KEY.



Domaine d'envoi en mode test

Sans domaine vérifié, Resend vous oblige à envoyer depuis onboarding@resend.dev.

L'adresse de destination peut être n'importe quelle adresse (la vôtre pour tester).

Pour envoyer depuis votre propre domaine (@mon-domaine.fr), il faut vérifier le domaine dans Resend → Domains.

D.2 Installer le SDK Resend

```
npm install resend
```

D.3 Créer l'API Route Vercel

Créez le dossier `api/` à la racine du projet (au même niveau que `src/`), puis le fichier `api/send-email.js` :

```

// api/send-email.js

// Ceci est une fonction serverless Vercel - elle s'exécute côté serveur

import { Resend } from 'resend';

const resend = new Resend(process.env.RESEND_API_KEY);

export default async function handler(req, res) {
  // Accepter uniquement les requêtes POST

  if (req.method !== 'POST') {
    return res.status(405).json({ error: 'Method not allowed' });
  }

  // Récupérer les données envoyées par le client React

  const { to, subject, html } = req.body;

  // Validation basique

  if (!to || !subject || !html) {
    return res.status(400).json({ error: 'Champs manquants : to, subject, html' });
  }

  try {
    const { data, error } = await resend.emails.send({
      from: 'KanbanRT <onboarding@resend.dev>', // à remplacer si domaine vérifié
      to, // ex: ['destinataire@exemple.com']
      subject, // ex: 'Nouvelle tâche assignée'
      html, // contenu HTML de l'e-mail
    });

    if (error) {
      return res.status(500).json({ error: error.message });
    }

    return res.status(200).json({ success: true, id: data.id });
  }
}

```

D.4 Appeler l'API Route depuis React

Maintenant, vous pouvez envoyer un e-mail depuis n'importe quel composant React en faisant un fetch vers `/api/send-email`. Voici un exemple à intégrer dans `ProfilePage.jsx` ou dans un composant dédié :

```
// Exemple d'utilisation dans un composant React
async function sendEmail() {

  const response = await fetch('/api/send-email', {

    method: 'POST',

    headers: { 'Content-Type': 'application/json' },

    body: JSON.stringify({

      to: ['destinataire@exemple.com'],

      subject: '📧 Nouvelle tâche KanbanRT',

      html: `

        <h1>Nouvelle tâche assignée !</h1>

        <p>La tâche <strong>Configurer Supabase</strong> vous a été
assignée.</p>

        <p>Statut : <em>À faire</em> · Priorité : <em>Haute</em></p>

        <a href='https://mon-kanban.vercel.app/dashboard'>Voir le tableau →</
a>

      `

    })),

  });

  const result = await response.json();

  if (result.success) {

    console.log('E-mail envoyé ! ID :', result.id);

  } else {

    console.error('Erreur :', result.error);

  }

}
```

D.5 Cas d'usage concret : notification d'assignation

Ajoutez dans TaskForm.jsx l'envoi automatique d'un e-mail lorsqu'une tâche est créée avec une date d'échéance :

```
// Dans handleSubmit() de TaskForm.jsx, après l'INSERT Supabase réussi :
if (!error && dueDate) {

  const formattedDate = new Date(dueDate).toLocaleDateString('fr-FR', {
    day: '2-digit', month: 'long', year: 'numeric'
  });

  await fetch('/api/send-email', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      to: [session.user.email], // envoyer au créateur de la tâche
      subject: `📅 Tâche créée : ${title}`,
      html: `
        <h2>Tâche créée avec succès</h2>
        <p><strong>Titre :</strong> ${title}</p>
        <p><strong>Priorité :</strong> ${priority}</p>
        <p><strong>Échéance :</strong> ${formattedDate}</p>
      `,
    }),
  });
}
```

⚡ En local : vite dev ne gère pas les API Routes

Les fonctions dans api/ sont exécutées par Vercel en production, pas par Vite en local.

Pour tester en local, installez le CLI Vercel : `npm install -g vercel`

Puis lancez : `vercel dev` (à la place de `npm run dev`)

Cette commande émule l'environnement Vercel complet sur votre machine.

Partie E — Tests & commit final (20 min)

E.1 Scénario de test complet

15. Lancez vercel dev (ou npm run dev pour les parties A–C).
16. Connectez-vous → la Navbar affiche vos liens et votre email.
17. Cliquez 'Mon profil' → le lien est surligné, la page s'affiche.
18. Modifiez votre nom complet → sauvegardez → message de confirmation.
19. Uploadez un avatar → l'image ronde s'affiche immédiatement.
20. Changez le mot de passe → déconnectez-vous → reconnectez-vous avec le nouveau.
21. Créez une tâche avec une date d'échéance → vérifiez la réception de l'e-mail.

E.2 Commit & push

```
git add .  
  
git commit -m "feat: add Navbar, ProfilePage with avatar upload and Resend email"  
  
git push origin main
```



Vercel en production

Vérifiez que RESEND_API_KEY est bien configurée dans Settings → Environment Variables sur Vercel.

Après le push, Vercel redéploie automatiquement. Testez l'envoi d'e-mail depuis l'URL Vercel.

Consultez les logs dans Vercel → Functions pour déboguer les erreurs de l'API Route.

Livrable de fin de séance

À déposer sur la plateforme de cours

1. URL du dépôt GitHub (doit contenir Navbar.jsx, ProfilePage.jsx et api/send-email.js)
2. URL Vercel avec les trois fonctionnalités fonctionnelles :
 - Navbar avec liens actifs et déconnexion
 - Page profil : modification du nom, changement de mot de passe, upload avatar
 - Envoi d'e-mail lors de la création d'une tâche avec échéance
3. Capture d'écran de la page profil avec avatar chargé
4. Capture de l'e-mail reçu dans votre boîte mail (ou screenshot du dashboard Resend)

Bonus — Pour aller plus loin

- Ajouter un champ 'bio' ou 'poste' dans les métadonnées utilisateur.
- Afficher l'avatar dans la Navbar à côté du nom (mini cercle 32px).
- Envoyer un e-mail de bienvenue automatiquement à l'inscription (onAuthStateChange + event === SIGNED_IN).
- Créer un template HTML plus riche pour l'e-mail de notification (couleurs, logo KanbanRT).
- Permettre la suppression du compte : `supabase.auth.admin.deleteUser()` depuis une API Route.
- Ajouter un bouton 'Mot de passe oublié' sur LoginPage via `supabase.auth.resetPasswordForEmail()`.